Nivass C R - 241801193
EXPERIMENT 10

Train a Generative Adversarial Network (GAN) to generate new images from a dataset. Evaluate the quality of the images generated using visual inspection and a quantitative metric like the Inception Score (IS) or Fréchet Inception Distance (FID). Suggested Dataset: LSUN Dataset

```
import tensorflow as tf
import matplotlib.pyplot as plt

tf.keras.utils.set_random_seed(42)

(x_train, _), _ = tf.keras.datasets.cifar10.load_data()
x_train = (x_train.astype("float32") - 127.5) / 127.5

dataset = tf.data.Dataset.from_tensor_slices(x_train)
dataset = dataset.shuffle(10000).batch(128).prefetch(tf.data.AUTOTUNE)

generator = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(100,)),
    tf.keras.layers.Dense(8 * 8 * 128),
    tf.keras.layers.Reshape((8, 8, 128)),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Conv2DTranspose(
        64, 4, strides=2, padding="same", activation="relu"
    ),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Conv2DTranspose(
        3, 4, strides=2, padding="same", activation="tanh"
    )
])

discriminator = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(32, 32, 3)),
    tf.keras.layers.Conv2D(64, 4, strides=2, padding="same"),
    tf.keras.layers.LeakyReLU(negative_slope=0.2),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(1)
])

g_optimizer = tf.keras.optimizers.Adam(0.0002, beta_1=0.5)
d_optimizer = tf.keras.optimizers.Adam(0.0002, beta_1=0.5)
loss_fn = tf.keras.losses.BinaryCrossentropy(from_logits=True)

@tf.function
def train_step(real_images):
    batch_size = tf.shape(real_images)[0]
    noise = tf.random.normal((batch_size, 100))

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        fake_images = generator(noise, training=True)

        real_output = discriminator(real_images, training=True)
        fake_output = discriminator(fake_images, training=True)

        gen_loss = loss_fn(tf.ones_like(fake_output), fake_output)
        real_loss = loss_fn(tf.ones_like(real_output), real_output)
        fake_loss = loss_fn(tf.zeros_like(fake_output), fake_output)
        disc_loss = real_loss + fake_loss

    gen_gradients = gen_tape.gradient(
        gen_loss, generator.trainable_variables
    )
    disc_gradients = disc_tape.gradient(
        disc_loss, discriminator.trainable_variables
    )

    g_optimizer.apply_gradients(
        zip(gen_gradients, generator.trainable_variables)
    )
    d_optimizer.apply_gradients(
        zip(disc_gradients, discriminator.trainable_variables)
    )

    return gen_loss, disc_loss
```

```
for epoch in range(5):
    gen_losses, disc_losses = [], []

    for batch in dataset:
        gl, dl = train_step(batch)
        gen_losses.append(float(gl))
        disc_losses.append(float(dl))

    print(
        f"Epoch {epoch + 1}/5 | "
        f"Generator Loss: {sum(gen_losses)/len(gen_losses):.4f} | "
        f"Discriminator Loss: {sum(disc_losses)/len(disc_losses):.4f}"
    )

noise = tf.random.normal((16, 100))
generated_images = generator(noise, training=False)

plt.figure(figsize=(7, 7))
for i in range(16):
    plt.subplot(4, 4, i + 1)
    img = (generated_images[i].numpy() + 1) / 2
    plt.imshow(img.clip(0, 1))
    plt.axis("off")

plt.suptitle("GAN Generated Images")
plt.tight_layout()
plt.show()
```

```
Epoch 1/5 | Generator Loss: 0.8638 | Discriminator Loss: 1.2330
Epoch 2/5 | Generator Loss: 0.6938 | Discriminator Loss: 1.4042
Epoch 3/5 | Generator Loss: 0.7060 | Discriminator Loss: 1.3873
Epoch 4/5 | Generator Loss: 0.7143 | Discriminator Loss: 1.3810
Epoch 5/5 | Generator Loss: 0.7261 | Discriminator Loss: 1.3618
```

